

Software Design Document (SDD)

Module: adc_driver - STM32F411 ADC Driver

Version: 1.0

Authors: Steven McClellan, Vania Leal, Leonardo Garcia, Melanie Picen

Item	Description
Source file	adc_driver.c
Main purpose	Initialize, configure, trigger, and read ADC1 regular and injected conversions on STM32F411.
Main public functions	adc_init(), adc_enableAdc(), adc_setChannel(), adc_setInjectedChannel(), adc_startSingleConversion(), adc_startContinuousConversion(), adc_startInjectedConversion(), adc_readData(), adc_readInjectedChannelData()

1. Introduction

1.1 Purpose

This document describes the design and architecture of the adc_driver module. The module provides a low-level ADC1 interface for the STM32F411 microcontroller. It enables the application to initialize the ADC peripheral, configure regular and injected channels, start conversions, and retrieve conversion results without manually manipulating ADC registers in the application layer.

1.2 Scope

The adc_driver module focuses on ADC1 operation. It directly accesses RCC and ADC1 memory-mapped registers to enable/reset the peripheral, configure software-triggered conversions, select a regular channel in SQ1, configure injected ranks in JSQR, and read DR/JDR conversion data. The implementation is polling-based and returns explicit status codes for invalid arguments, busy states, and timeouts.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
ADC	Analog-to-Digital Converter. Converts analog voltage into a digital value.
ADC1	ADC peripheral instance used by this driver.
RCC	Reset and Clock Control block used to enable and reset ADC1.
APB2	Peripheral bus that clocks ADC1 on STM32F411.
CR1/CR2	ADC control registers. CR2 contains ADON, CONT, SWSTART, JSWSTART, EXTEN, EOCS, and ALIGN.
SQR1/SQR3	Regular sequence registers. This driver sets sequence length to 1 and places the selected channel in SQ1.
JSQR	Injected sequence register. Stores injected channel rank fields.
DR	Regular data register. Holds the regular conversion result.
JDR1-JDR4	Injected data registers for injected ranks 1 through 4.
EOC	End Of Conversion flag for regular conversions.
JEOC	Injected End Of Conversion flag.
SWSTART	Software start bit for regular conversion.
JSWSTART	Software start bit for injected conversion.
SDD	Software Design Document.

1.4 Requirements Traceability

Req. ID	Function	Requirement Statement
FR-1	adc_init()	Enable ADC1 clock on APB2 and reset/release the ADC1 peripheral.

FR-2	<code>adc_enableAdc()</code>	Validate the ADC instance, power on ADC1 through ADON, select software triggering, right alignment, and final EOC behavior.
FR-3	<code>adc_setChannel()</code>	Validate instance/channel and configure a one-conversion regular sequence with the selected channel in SQ1.
FR-4	<code>adc_setInjectedChannel()</code>	Validate instance/channel/rank and write the selected channel into the requested injected rank field.
FR-5	<code>adc_startSingleConversion()</code>	Clear EOC, wait for previous software start to clear, then trigger a single regular conversion.
FR-6	<code>adc_startContinuousConversion()</code>	Enable continuous conversion mode and trigger regular conversion when the ADC is not busy.
FR-7	<code>adc_startInjectedConversion()</code>	Trigger injected conversion by setting JSWSTART when the injected group is not busy.
FR-8	<code>adc_readData()</code>	Poll EOC with timeout, read the regular conversion result from DR, and clear EOC.
FR-9	<code>adc_readInjectedChannelData()</code>	Poll JEOC with timeout and read the requested JDR register.

Non-functional requirements: no dynamic memory allocation; direct register access; polling-based synchronization; bounded waits using software timeouts; status-code based error reporting; no HAL middleware dependency inside this source file.

2. System Architecture

2.1 High-Level Design

The ADC driver is a low-level hardware abstraction layer. The application calls the public API. The driver validates parameters and interacts with RCC and ADC1 registers. The hardware layer performs the actual sampling and stores conversion results in ADC data registers.

Layer	Files / Components	Description
Application Layer	<code>main.c</code> or higher-level modules	Calls the ADC driver API and consumes conversion results.
ADC Driver Layer	<code>adc_driver.c</code> / <code>adc_driver.h</code>	Implements ADC1 initialization, channel selection, conversion triggering, and data readback.
Register Layer	ADC1 and RCC memory-mapped registers	Provides MODER-independent ADC register access through the STM32 register definitions and direct RCC addresses.
Hardware Layer	STM32F411 ADC1 and analog input pins	Performs analog sampling and conversion.

2.2 Class Diagram

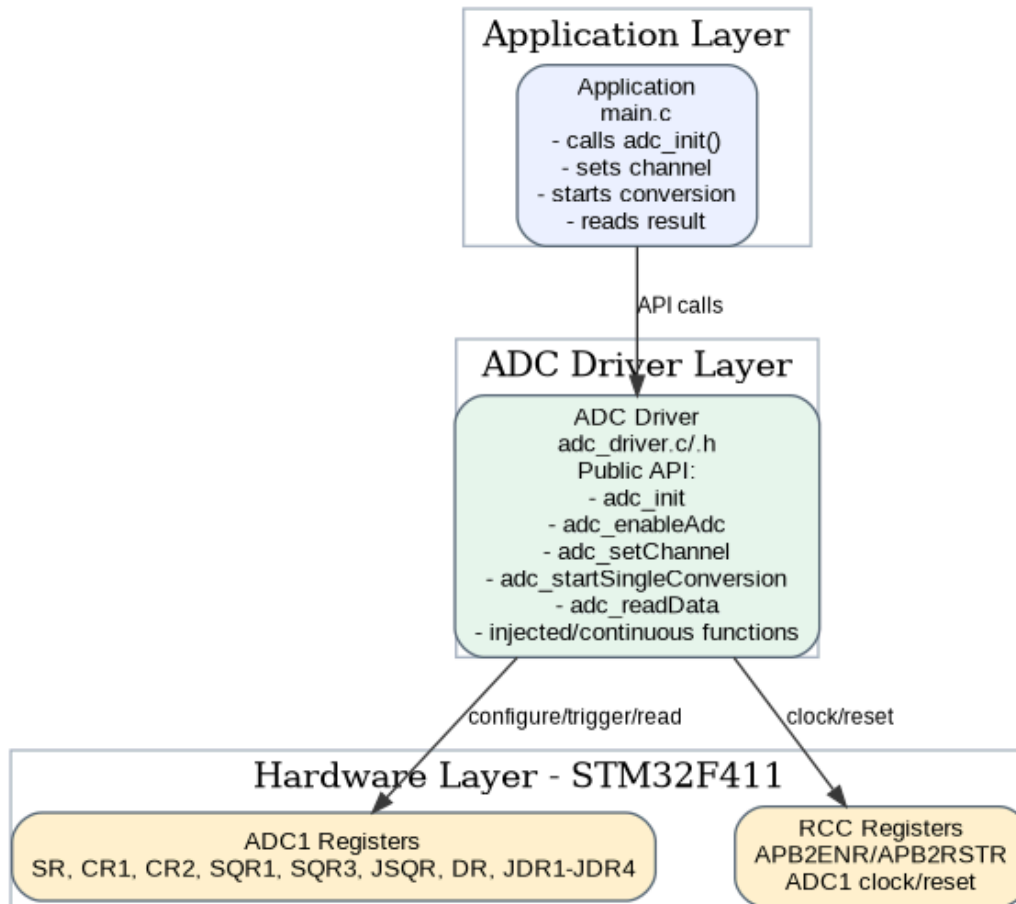


Figure 1. Class Diagram - `adc_driver` module.

2.3 Module Interfaces

Interface	Description
Public API	<code>adc_init()</code> , <code>adc_enableAdc()</code> , <code>adc_setChannel()</code> , <code>adc_setInjectedChannel()</code> , <code>adc_startSingleConversion()</code> , <code>adc_startContinuousConversion()</code> , <code>adc_startInjectedConversion()</code> , <code>adc_readData()</code> , <code>adc_readInjectedChannelData()</code> .
External types	<code>Adc_Status_t</code> , <code>Adc_Instance_t</code> , <code>Adc_Channel_t</code> , <code>Adc_InjectedRank_t</code> , and ADC1 register definition type are expected from <code>adc_driver.h</code> .
Hardware dependencies	RCC APB2 enable/reset registers and ADC1 SR, CR2, SQR1, SQR3, JSQR, DR, and JDRx registers.

3. Detailed Design

3.1 Module Behavior

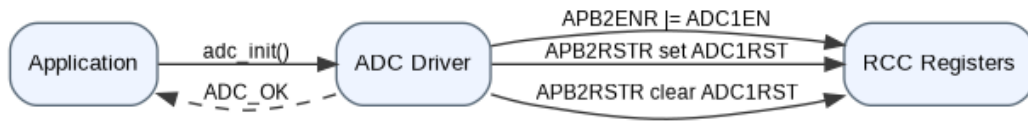


Figure 2. Sequence Diagram - `adc_init`.

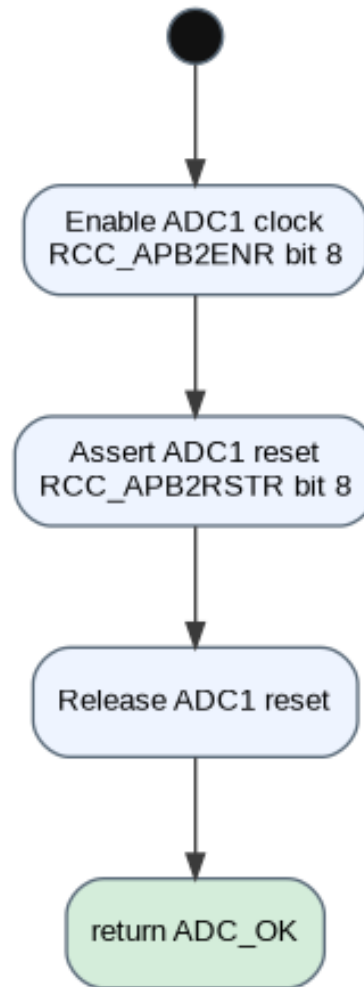


Figure 3. Activity Diagram - `adc_init`.

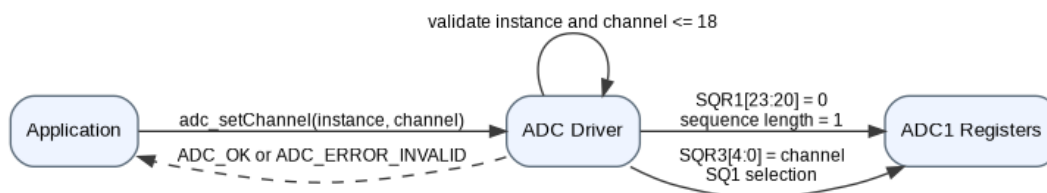


Figure 4. Sequence Diagram - `adc_setChannel`.

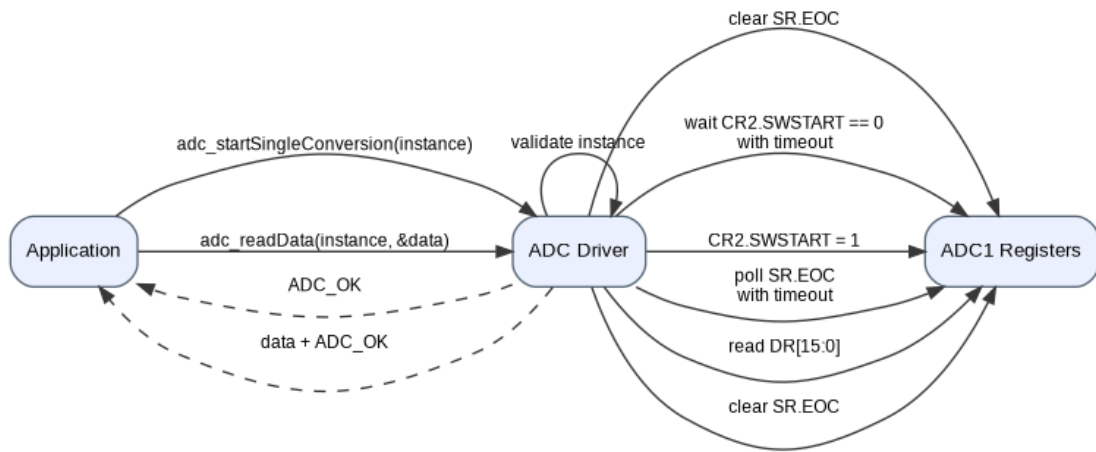


Figure 5. Sequence Diagram - regular conversion flow.

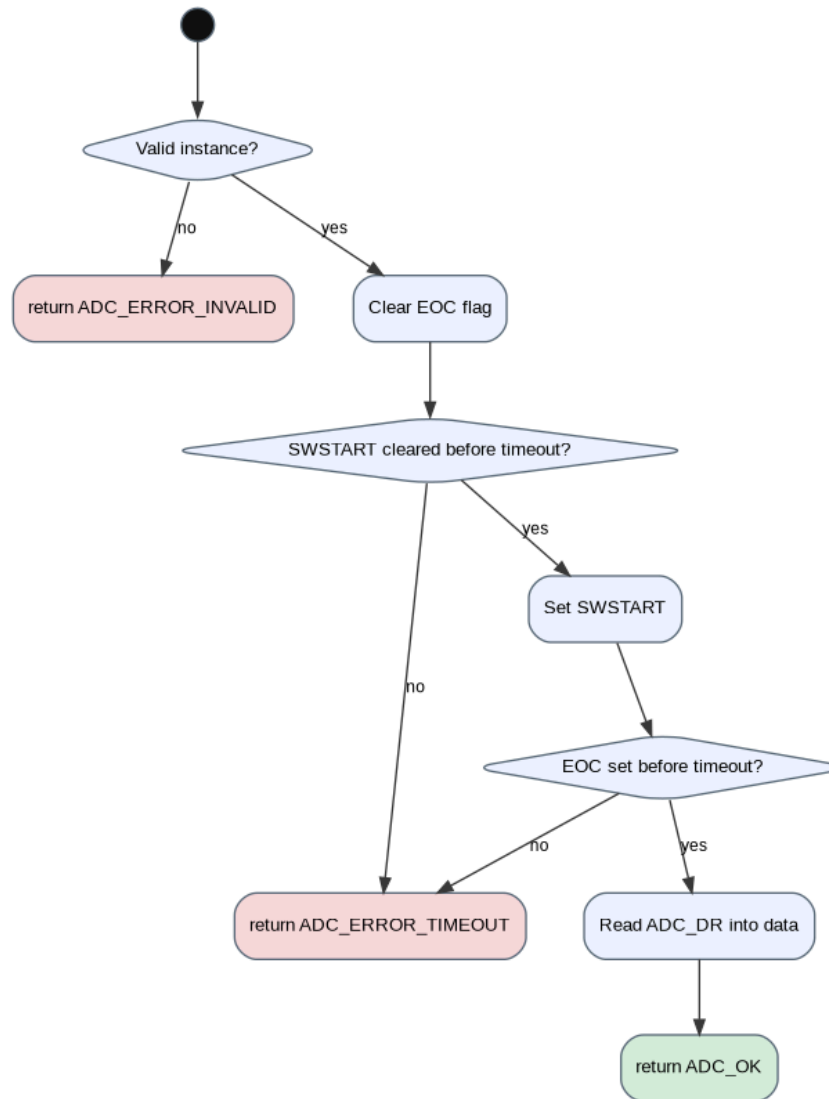


Figure 6. Activity Diagram - single regular conversion and read.

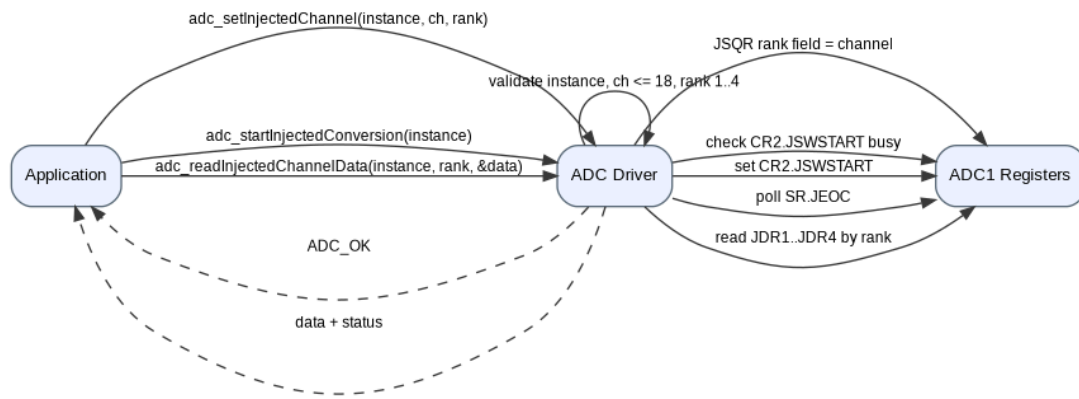


Figure 7. Sequence Diagram - injected channel flow.

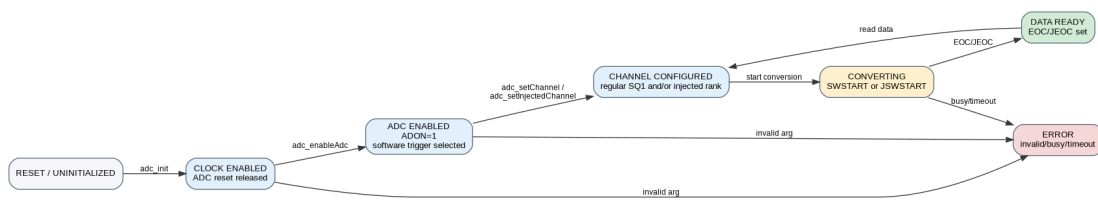


Figure 8. State Diagram - ADC driver lifecycle.

3.2 adc_driver Module

The driver begins by enabling the ADC1 clock through RCC_APB2ENR and performing a reset pulse through RCC_APB2RSTR. After initialization, `adc_enableAdc()` powers the ADC by setting ADON in CR2. It also forces software triggering by clearing EXTEN, sets EOCS to sequence-end behavior, and selects right alignment by clearing ALIGN.

3.3 Regular Channel Configuration

`adc_setChannel()` configures a single regular conversion. It clears SQR1[23:20], which selects a sequence length of one conversion, then clears and writes SQR3[4:0] so that the selected channel becomes SQ1. The implementation validates that the channel is between 0 and 18.

3.4 Regular Conversion Flow

`adc_startSingleConversion()` clears EOC, waits until SWSTART is not set, and then sets SWSTART to trigger a software conversion. `adc_readData()` polls EOC with a fixed timeout, reads the lower 16 bits of ADC_DR into the caller-provided output pointer, and clears EOC manually. Continuous conversion follows the same regular conversion path but sets CR2.CONT before SWSTART.

3.5 Injected Conversion Flow

`adc_setInjectedChannel()` writes the selected channel to one of four JSQR rank fields. `adc_startInjectedConversion()` checks JSWSTART to avoid starting while another injected conversion is active, then sets JSWSTART. `adc_readInjectedChannelData()` polls JEOC and reads JDR1, JDR2, JDR3, or JDR4 depending on the requested rank.

4. Application Programming Interfaces

4.1 Data Types and Macros

Macro / Type	Value / Meaning	Notes
RCC_BASE	0x40023800u	Base address used to access RCC registers directly.
ADC1_OFFSET	8u	Bit position used for ADC1 clock enable and reset in APB2 registers.

RCC_APB2ENR_REG	*(RCC_BASE + 0x44)	APB2 peripheral clock enable register.
RCC_APB2RSTR_REG	*(RCC_BASE + 0x24)	APB2 peripheral reset register.
Adc_Status_t	ADC_OK, ADC_ERROR_INVALID, ADC_ERROR_TIMEOUT, ADC_ERROR_BUSY	Status return type inferred from the implementation.
Adc_Instance_t	ADC instance selector	Validated against ADC_INSTANCE_MAX. Current source uses ADC1 registers directly.
Adc_Channel_t	0..18	ADC channel number accepted by regular/injected configuration.
Adc_InjectedRank_t	1..4	Injected rank used to select JSQR field and JDRx readback.

4.2 Error Codes

Code	Notes / Definition
ADC_OK	Operation completed successfully.
ADC_ERROR_INVALID	Invalid instance, channel, injected rank, or unsupported parameter.
ADC_ERROR_TIMEOUT	Polling loop expired before the expected flag/state was reached.
ADC_ERROR_BUSY	A conversion start was requested while the corresponding conversion start bit was already active.

adc_init

Field	Definition
Function Name	adc_init
Syntax	Adc_Status_t adc_init(void)
Parameters (in)	None.
Parameters (out)	None.
Return value	ADC_OK.
Description	Enables ADC1 clock and resets/releases ADC1 through RCC APB2 registers.
Constraints	Only initializes/reset-enables ADC1. It does not configure GPIO analog mode or channels.
Available via	adc_driver.h

adc_enableAdc

Field	Definition
Function Name	adc_enableAdc
Syntax	Adc_Status_t adc_enableAdc(Adc_Instance_t instance)
Parameters (in)	instance: ADC instance selector.
Parameters (out)	None.
Return value	ADC_OK or ADC_ERROR_INVALID.
Description	Powers on ADC1 using CR2.ADON and configures software trigger, sequence-end EOC, and right alignment.
Constraints	The instance parameter is validated but the implementation uses ADC1 directly.
Available via	adc_driver.h

adc_setChannel

Field	Definition
Function Name	adc_setChannel
Syntax	Adc_Status_t adc_setChannel(Adc_Instance_t instance, Adc_Channel_t channel)
Parameters (in)	instance and channel.
Parameters (out)	None.
Return value	ADC_OK or ADC_ERROR_INVALID.
Description	Configures a one-channel regular conversion sequence and places channel in SQ1.
Constraints	Only one regular channel is configured; multi-channel scan sequence is not implemented.
Available via	adc_driver.h

adc_setInjectedChannel

Field	Definition
Function Name	adc_setInjectedChannel

Syntax	Adc_Status_t adc_setInjectedChannel(Adc_Instance_t instance, Adc_Channel_t channel, Adc_InjectedRank_t rank)
Parameters (in)	instance, channel, and rank.
Parameters (out)	None.
Return value	ADC_OK or ADC_ERROR_INVALID.
Description	Writes channel into the JSQR field corresponding to rank 1..4.
Constraints	The implementation writes rank fields but does not explicitly update JSQR injected sequence length bits.
Available via	adc_driver.h

adc_startSingleConversion

Field	Definition
Function Name	adc_startSingleConversion
Syntax	Adc_Status_t adc_startSingleConversion(Adc_Instance_t instance)
Parameters (in)	instance.
Parameters (out)	None.
Return value	ADC_OK, ADC_ERROR_INVALID, or ADC_ERROR_TIMEOUT.
Description	Starts a software-triggered regular conversion by setting CR2.SWSTART after clearing EOC and checking previous SWSTART state.
Constraints	Polling timeout is fixed in software.
Available via	adc_driver.h

adc_startContinuousConversion

Field	Definition
Function Name	adc_startContinuousConversion
Syntax	Adc_Status_t adc_startContinuousConversion(Adc_Instance_t instance)
Parameters (in)	instance.
Parameters (out)	None.
Return value	ADC_OK, ADC_ERROR_INVALID, or ADC_ERROR_BUSY.
Description	Enables CR2.CONT and sets SWSTART to begin continuous regular conversion.
Constraints	Does not provide a stop function or clear CONT in this source file.
Available via	adc_driver.h

adc_startInjectedConversion

Field	Definition
Function Name	adc_startInjectedConversion
Syntax	Adc_Status_t adc_startInjectedConversion(Adc_Instance_t instance)
Parameters (in)	instance.
Parameters (out)	None.
Return value	ADC_OK, ADC_ERROR_INVALID, or ADC_ERROR_BUSY.
Description	Starts an injected conversion by setting CR2.JSWSTART.
Constraints	Busy check only observes JSWSTART state.
Available via	adc_driver.h

adc_readData

Field	Definition
Function Name	adc_readData
Syntax	Adc_Status_t adc_readData(Adc_Instance_t instance, uint16_t *data)
Parameters (in)	instance.
Parameters (out)	data: receives ADC_DR[15:0].
Return value	ADC_OK, ADC_ERROR_INVALID, or ADC_ERROR_TIMEOUT.
Description	Polls EOC, reads the regular data register, and clears EOC.
Constraints	The implementation does not check data for NULL before writing through it.
Available via	adc_driver.h

adc_readInjectedChannelData

Field	Definition
-------	------------

Function Name	adc_readInjectedChannelData
Syntax	Adc_Status_t adc_readInjectedChannelData(Adc_Instance_t instance, Adc_InjectedRank_t rank, uint16_t *data)
Parameters (in)	instance and rank.
Parameters (out)	data: receives selected JDRx[15:0].
Return value	ADC_OK, ADC_ERROR_INVALID, or ADC_ERROR_TIMEOUT.
Description	Polls JEOC and reads the injected data register selected by rank.
Constraints	The implementation does not check data for NULL before writing through it.
Available via	adc_driver.h

5. Implementation Notes and Constraints

Topic	Note
ADC1-only implementation	Although an instance parameter is present, all functions access ADC1 directly. This matches STM32F411 single-ADC use but should be documented.
Pointer validation	adc_readData() and adc_readInjectedChannelData() should validate data != NULL before dereferencing.
GPIO analog configuration	The driver configures ADC registers only. The corresponding GPIO pin must be configured as analog by another module.
Single regular channel	adc_setChannel() forces a one-conversion sequence and does not implement multi-channel scan configuration.
Injected sequence length	Injected rank fields are written, but JSQR sequence length bits are not explicitly configured.
Polling model	Conversions are synchronized using polling loops with fixed timeout values.
Unused CR1 bit macros	Several ADC_CR_* macros are defined but are not used by the current implementation.

6. Future Enhancements

- Add NULL pointer checks in adc_readData() and adc_readInjectedChannelData().
- Use the instance parameter to support multiple ADC instances where available, or remove it if ADC1-only behavior is intended.
- Add GPIO analog-mode configuration guidance or helper integration.
- Add multi-channel regular sequence support using SQR1/SQR2/SQR3 and scan mode.
- Explicitly configure injected sequence length bits in JSQR.
- Add a function to stop continuous conversion and clear CONT.
- Replace magic bit numbers with named macros for CR2/SR flags.
- Add unit or hardware-in-loop tests for invalid parameters, timeout handling, busy handling, and data readback.